

SoPC Abgabedokument

Florian Breitwieser & Jakob Zimmermann

8. Mai 2026

Inhaltsverzeichnis

1	Addressierungsbeispiel	3
1.1	Einleitung	3
1.2	Aufgabenstellung	3
1.3	Addr_Slave Code	4
1.4	Addr_Slave Testbench	5
1.5	Master Architektur und Code	7
1.6	Testbench	11
1.7	Metastabilität	13
1.7.1	Ursachen	13
1.7.2	Auswirkungen	13
1.7.3	Vermeidung	13
2	UART TX	14
3	Aufgabe 4: UART RX	22

Abbildungsverzeichnis

1	Simulation des Address-Slave mit Testbench	6
2	Blockschaltbild des Address-Master und Slave-Anbindung	7
3	Zustandsdiagramm des Address-Master	10
4	Simulation des Address-Master mit Slave	12
5	UART TX Blockdiagramm	14
6	UART TX Statemaschine	15
7	UART TX Simulation 1	21
8	UART TX Simulation 2	21
9	UART TX Simulation 3	22
10	UART RX Blockdiagramm	23

11	GTKWave-Simulation des UART RX Moduls: Verlauf der Signale <code>state</code> , <code>uart_in</code> und <code>led</code>	30
----	--	----

1 Adressierungsbeispiel

1.1 Einleitung

Dieses Dokument beschreibt die Umsetzung des Adressierungsbeispiels als Teil der Aufgabe 3. Es fasst die Architektur des Address-Slave und des Address-Master zusammen, dokumentiert die Simulation und beschreibt Metastabilitätseffekte.

1.2 Aufgabenstellung

Ziel ist die Implementierung eines adressierbaren LED-Slave-Moduls für das iceduino-Board und eines synthetisierbaren Masters, der die LEDs in einem Lauflicht ansteuert. Die Abgabeanforderungen umfassen:

- Slave Code und erläuternder Text zur Funktion
- Slave Testbench Code und erläuternder Text zur Funktion, inklusive asserts und Hinweis auf nicht-synthetisierbaren Testbench-Code
- Simulation Slave mit Testbench (Screenshot + Analyse)
- Master Blockschaltbild inkl. Slave-Anbindung
- Master Code
- Master Simulation mit Slave
- Erläuterung der Simulationsergebnisse und Master-Statemachine
- Metastabilitätsbetrachtung: Ursache, Auswirkung und Vermeidung

1.3 Addr_Slave Code

Der folgende Code zeigt das Low-Level-Verhalten des Slave-Moduls. Es verarbeitet den Reset, liest die Adresse und schreibt bei gesetztem `we` das Datensignal `d` in das entsprechende LED-Register.

```
1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity addr_slave is
6      port(
7          clk: in std_logic;
8          we : in std_logic;
9          d  : in std_logic;
10         rst : in std_logic;
11         addr : in std_logic_vector(2 downto 0);
12         leds : out std_logic_vector(7 downto 0)
13     );
14 end entity;
15
16 architecture addr_slave_a of addr_slave is
17 begin
18     process(clk, rst) is
19     begin
20         if rst = '1' then
21             leds <= (others => '0');
22         elsif rising_edge(clk) then
23             if we = '1' then
24                 leds(to_integer(unsigned(addr))) <= d;
25             end if;
26         end if;
27     end process;
28 end architecture addr_slave_a;
```

Bei steigender Flanke des Taktsignals `clk` und aktivem Reset (`rst = '1'`) werden alle LEDs gelöscht. Wenn `we` gesetzt ist, wird das Datenbit `d` an der Position geschrieben, die durch die Adresse `addr` bestimmt wird. Die LEDs werden als 8-Bit-Vektor ausgegeben, wobei jedes Bit eine LED repräsentiert.

1.4 Addr_Slave Testbench

Die Testbench prüft die korrekte Adressierung des Slave-Moduls für mehrere Adressen und enthält Assert-Anweisungen für die LED-Ausgabe.

```
1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity addr_slave_tb is
6  end entity;
7
8  architecture addr_slave_tb_a of addr_slave_tb is
9      signal clk : std_logic := '0';
10     signal we : std_logic := '0';
11     signal d : std_logic := '0';
12     signal reset : std_logic := '0';
13     signal addr : std_logic_vector(2 downto 0) := (others => '0');
14     signal leds : std_logic_vector(7 downto 0) := (others => '0');
15 begin
16     clk <= not clk after 10 ns; -- 50 MHz
17
18     addr_slave_instance : entity work.addr_slave
19         port map(
20             clk => clk,
21             we => we,
22             d => d,
23             reset => reset,
24             addr => addr,
25             leds => leds
26         );
27
28     process is
29     begin
30         reset <= '1';
31         wait for 30 ns;
32         reset <= '0';
33
34         addr <= (others => '0');
35         wait for 10 ns;
36         d <= '1';
37         we <= '1';
38
39         wait for 80 ns;
40         assert (leds = "00000001") report "R1: LEDS falsch" severity error;
41         assert (addr = "000") report "R1: b0 falsch" severity error;
42         report "R1: OK" severity note;
43         we <= '0';
44         wait for 1 ns;
45         d <= '0';
```

```

46
47     wait for 200 ns;
48     addr <= (others => '1');
49     wait for 10 ns;
50     d <= '1';
51     we <= '1';
52
53     wait for 80 ns;
54     assert (leds = "10000001") report "R2: LEDS falsch" severity error;
55     assert (addr = "111") report "R2: addr falsch" severity error;
56     report "R2: OK" severity note;
57     we <= '0';
58     wait for 1 ns;
59     d <= '0';
60
61     wait for 200 ns;
62     addr <= "010";
63     wait for 10 ns;
64     d <= '1';
65     we <= '1';
66
67     wait for 80 ns;
68     assert (leds = "10000101") report "R3: LEDS falsch" severity error;
69     report "R3: OK" severity note;
70     we <= '0';
71     wait for 200 ns;
72     d <= '0';
73 end process;
74 end architecture addr_slave_tb_a;

```

Alle Asserts konnten erfolgreich durchlaufen werden. Der addr-slave funktioniert einwandfrei.

Alle 'wait' und 'after' Anweisungen sind nicht-synthetisierbar und dienen ausschließlich der Simulation.

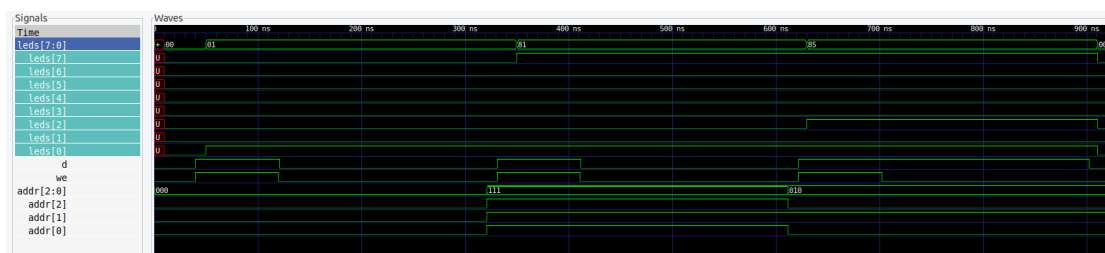


Abbildung 1: Simulation des Address-Slave mit Testbench

Die Simulation zeigt die korrekte Adressierung und LED-Ausgabe für die getesteten Adressen. Bei Adresse '000' leuchtet die erste LED, bei '111' die letzte LED, und bei '010' die dritte LED. Alle Asserts bestätigen die erwarteten Ergebnisse.

1.5 Master Architektur und Code

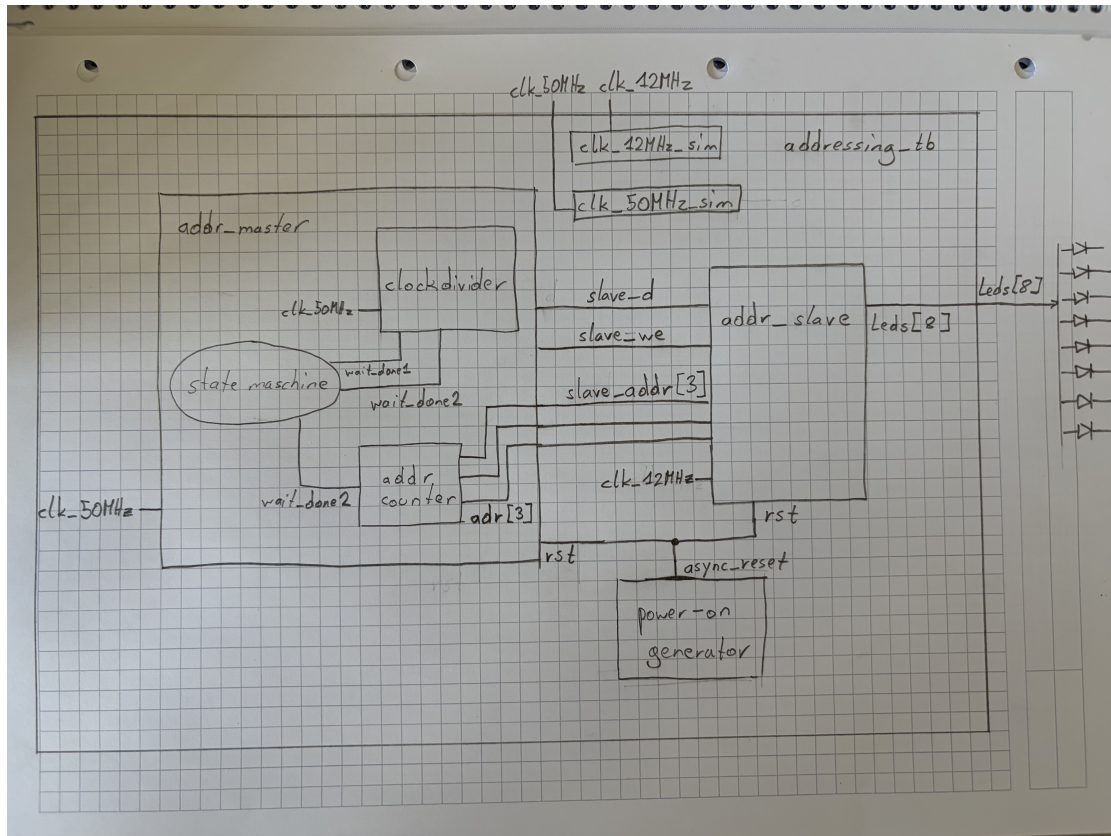


Abbildung 2: Blockschaltbild des Address-Master und Slave-Anbindung

Das Bild zeigt die Zusammenschaltung des Address-Master mit dem Address-Slave. Der Master generiert die Signale **adr**, **we** und **d**, die direkt an den Slave weitergegeben werden. Ein Power-On-Reset-Generator sorgt für einen definierten Reset-Zustand nach FPGA-Neustart.

Der **addr_master** erzeugt die Steuerungsimpulse für den Slave. Er durchläuft die Adressen 000 bis 111 und schaltet jeweils eine LED ein, wartet dann eine definierte Verzögerungszeit, schaltet ab und geht zur nächsten Adresse.

```

1  entity addr_master is
2      generic (
3          variable_delay : integer := 12500000
4      );
5      port(
6          clk : in std_logic;
7          rst : in std_logic;
8          adr : out std_logic_vector(2 downto 0);

```

```

9         we : out std_logic;
10        d : out std_logic
11    );
12 end entity;
13
14 architecture addr_master_a of addr_master is
15     type state_type is (START, LON, LOFF);
16     signal state : state_type := START;
17     signal cnt : integer range 0 to 12500000 := 0;
18     signal adr_reg : unsigned(2 downto 0) := "000";
19     signal delay_cnt : integer range 0 to variable_delay := 0;
20     signal slave_we : std_logic := '0';
21     signal slave_d : std_logic := '0';
22     signal delay_cnt2 : integer range 0 to 20000 := 0;
23     signal wait_done1 : std_logic := '0';
24     signal wait_done2 : std_logic := '0';
25 begin
26
27     process (clk, rst)
28     begin
29
30         if rst = '1' then
31             state <= START;
32             slave_we <= '0';
33             slave_d <= '0';
34             wait_done1 <= '0';
35             wait_done2 <= '0';
36         elsif rising_edge(clk) then
37             case state is
38                 when START =>
39                     state <= LON;
40                     wait_done1 <= '0';
41                     wait_done2 <= '0';
42                 when LON =>
43                     slave_d <= '1';
44                     if delay_cnt = 0 then
45                         slave_we <= '1';
46                         delay_cnt <= delay_cnt + 1;
47                     elsif delay_cnt = 10 then
48                         slave_we <= '0';
49                         delay_cnt <= delay_cnt + 1;
50                     elsif delay_cnt = variable_delay then
51                         delay_cnt <= 0;
52                         wait_done1 <= '1';
53                     else
54                         delay_cnt <= delay_cnt + 1;
55                     end if;
56
57                     if wait_done1 = '1' then
58                         state <= LOFF;

```



```

59         wait_done1 <= '0';
60     end if;
61 when LOFF =>
62     slave_d <= '0';
63     slave_we <= '1';
64
65     if delay_cnt2 = 10 then
66         delay_cnt2 <= 0;
67         slave_we <= '0';
68         wait_done2 <= '1';
69     else
70         delay_cnt2 <= delay_cnt2 + 1;
71     end if;
72
73     if wait_done2 = '1' then
74         if adr_reg = "111" then
75             adr_reg <= "000";
76         else
77             adr_reg <= adr_reg + 1;
78         end if;
79
80         state <= LON;
81         wait_done2 <= '0';
82     end if;
83 end case;
84 end if;
85 end process;
86
87 adr <= std_logic_vector(adr_reg);
88 d <= slave_d;
89 we <= slave_we;
90 end architecture addr_master_a;

```

Der Master verwendet einen Zustandsautomaten mit den Zuständen:

- **START**
- **LON** (LED einschalten)
- **LOFF** (LED ausschalten)

Das folgende Zustandsdiagramm zeigt die Übergänge zwischen den Zuständen.

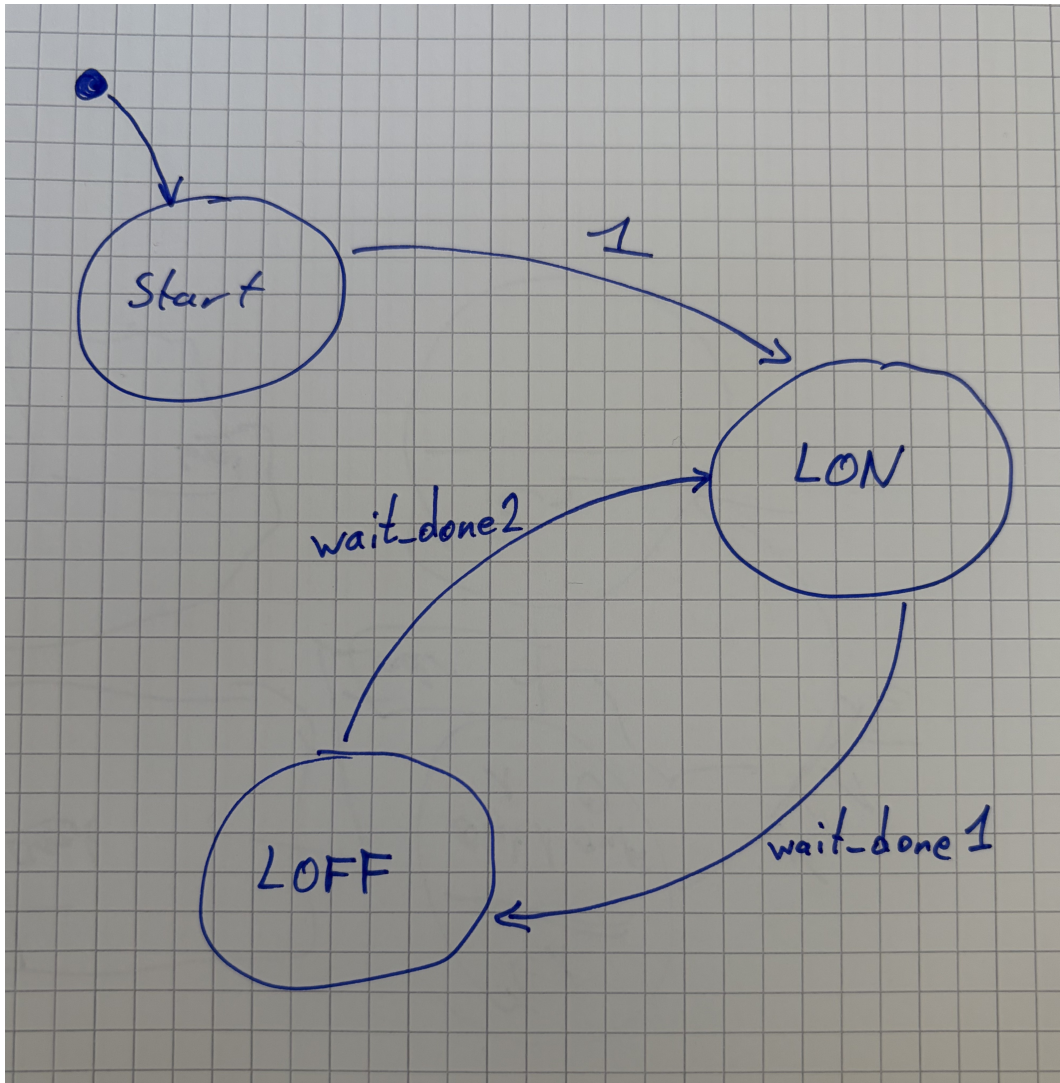


Abbildung 3: Zustandsdiagramm des Address-Master

1.6 Testbench

Die Testbench `addressing_tb.vhd` implementiert die Addressing-Simulation mit verkürztem Delay.

```
1  entity addressing_tb is
2      generic (
3          variable_delay : integer := 500
4      );
5
6      port(
7          clk_12MHz : in std_logic;
8          clk_50MHz : in std_logic;
9          leds : out std_logic_vector(7 downto 0)
10     );
11 end entity;
12
13 architecture addressing_tb_a of addressing_tb is
14     signal async_reset : std_logic := '1';
15     signal slave_we : std_logic := '0';
16     signal slave_d : std_logic := '0';
17     signal slave_addr : std_logic_vector(2 downto 0) := "000";
18     signal clk_12MHz_sim : std_logic := '0';
19     signal clk_50MHz_sim : std_logic := '0';
20     signal counter : unsigned(7 downto 0) := (others => '0');
21 begin
22
23     clk_12MHz_sim <= not clk_12MHz_sim after 41.666666 ns;
24     clk_50MHz_sim <= not clk_50MHz_sim after 10 ns;
25
26     process(clk_12MHz_sim) is
27         variable cnt : unsigned(7 downto 0) := (others => '0');
28     begin
29         if rising_edge(clk_12MHz_sim) then
30             if cnt < 255 then
31                 cnt := cnt + 1;
32                 counter <= cnt;
33                 async_reset <= '1';
34             else
35                 counter <= cnt;
36                 async_reset <= '0';
37             end if;
38         end if;
39     end process;
40
41     addr_master_instance : entity work.addr_master
42     generic map(
43         variable_delay => variable_delay
44     )
45     port map(
```

```

46         clk => clk_50MHz_sim,
47         rst => async_reset,
48         adr => slave_addr,
49         we => slave_we,
50         d => slave_d
51     );
52
53     addr_slave_instance : entity work.addr_slave
54     port map(
55         clk => clk_12MHz_sim,
56         we => slave_we,
57         d => slave_d,
58         rst => async_reset,
59         addr => slave_addr,
60         leds => leds
61     );
62
63
64     end architecture addressing_tb_a;

```

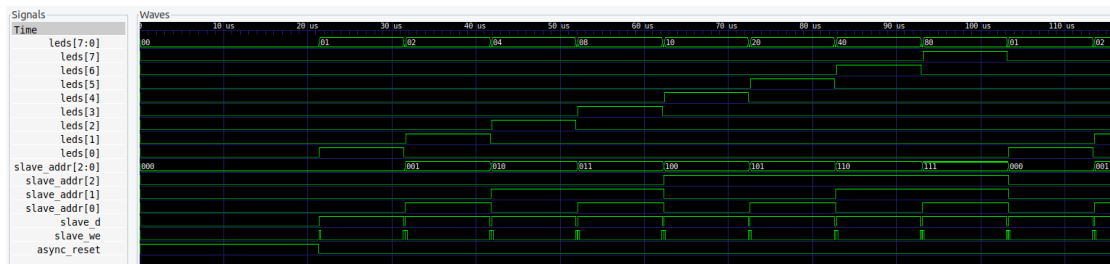


Abbildung 4: Simulation des Address-Master mit Slave

Die Simulation zeigt die korrekte Funktionalität des Masters und Slaves. Der Master durchläuft die Adressen von '000' bis '111', schaltet die entsprechenden LEDs ein und aus.

1.7 Metastabilität

Flip-Flops können metastabil werden, wenn asynchrone Eingangssignale sehr nahe an der aktiven Flanke des Taktes wechseln. In diesem Zustand bleibt das Flip-Flop für eine unbestimmte Zeit in einem undefinierten Spannungsbereich, was zu Glitches, Fehlern oder falschen Datenweiterleitungen führen kann.

1.7.1 Ursachen

- asynchrone Signaländerungen nahe am Taktflankenzeitpunkt
- nicht eingehaltene Setup- und Hold-Zeiten
- fehlende Synchronisation zwischen verschiedenen Taktdomänen

1.7.2 Auswirkungen

- fehlerhafte Signalausgabe
- unvorhersehbares Verhalten von State Machines
- sporadische Systemausfälle

1.7.3 Vermeidung

- Synchronisationsstufen (z. B. zwei Flip-Flops hintereinander)
- ausreichende Setup- und Hold-Zeiten
- Verwendung von Taktdomänen-Grenzen und Cross-Clock-Handshake

2 UART TX

Das folgende Blockdiagramm zeigt die Struktur des UART TX Moduls. Es besteht aus einem Hauptmodul `uart_tx` und einem Testbench-Modul `uart_tx_tb`. Das Hauptmodul implementiert die UART TX Funktionalität, während das Testbench-Modul zur Simulation und Verifikation der Funktionalität des Hauptmoduls dient. Der `clk_ext`-extern Eingang zeigt den tatsächlichen Clock-Eingang, falls eine externe Clock verwendet werden würde. Da wir jedoch die simulierte Clock der Testbench verwenden, ist dieser Eingang nur symbolisch. Die Signale `rst`, `clk 12MHz`, `byte_in` und `we` werden von der Testbench erzeugt, um die Funktionalität des UART TX Moduls zu testen.

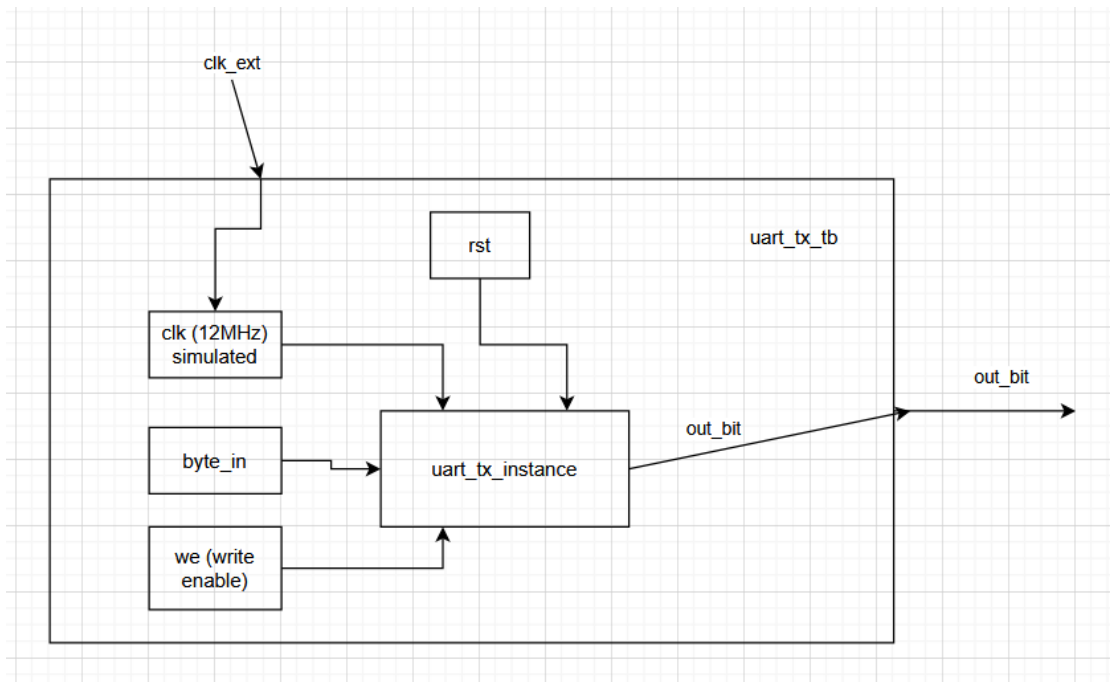


Abbildung 5: UART TX Blockdiagramm

Die Statesmaschine des UART TX Moduls besteht aus zwei Hauptzuständen: IDLE und SENDING. Der Automat ist deterministisch.

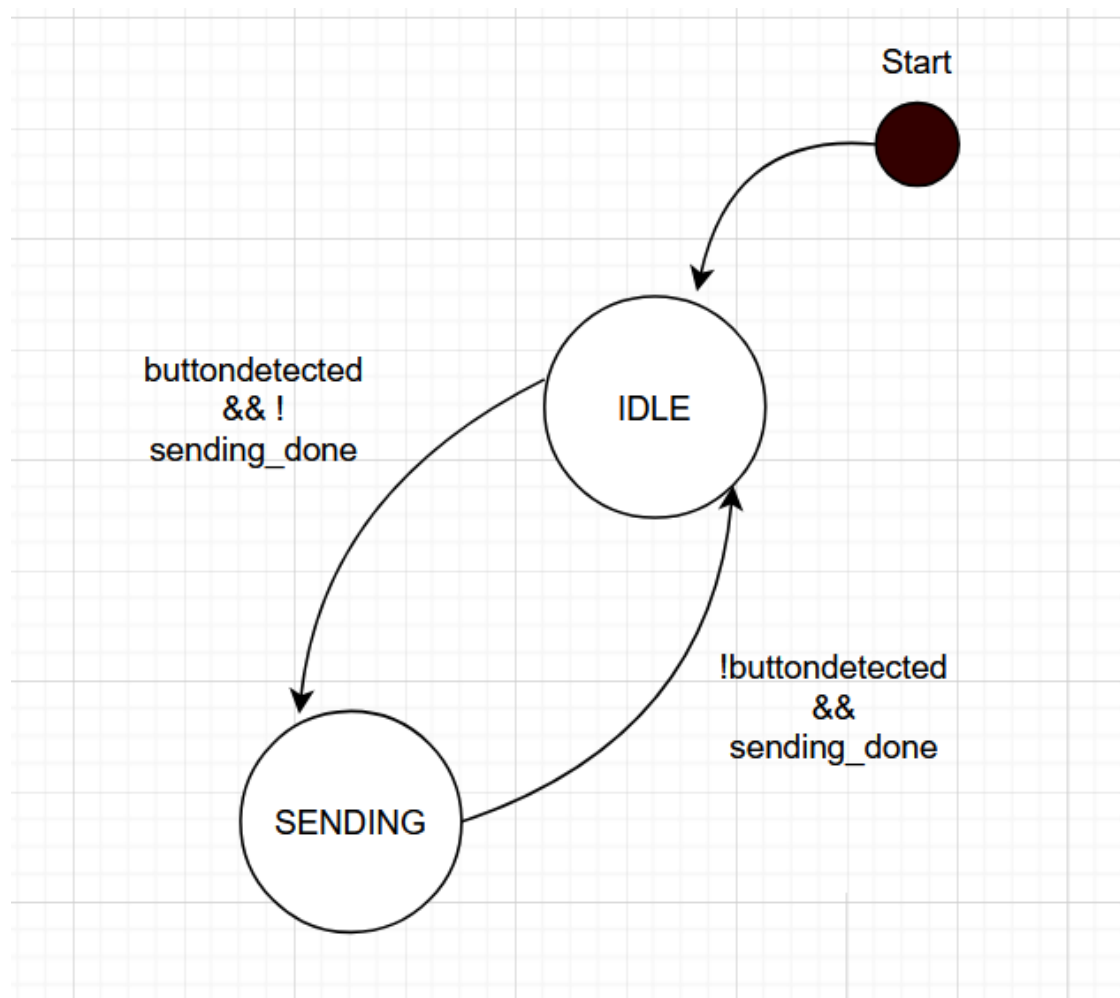


Abbildung 6: UART TX Statesmaschine

```

1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity uart_tx is
6      port(
7          clk : in std_logic;
8          byte_in : in std_logic_vector(7 downto 0);
9          we : in std_logic;
10         out_bit : out std_logic;
11         rst : in std_logic
12     );
13 end entity;
14
15
16 architecture uart_tx_a of uart_tx is
17     type state_type is (IDLE, SENDING);
18     signal state : state_type := IDLE;
19     signal symbol_cnt : integer := 0;
20     constant baud_rate : integer := 9600;
21     constant clk_speed : integer := 12000000;
22     constant bit_period : integer := clk_speed / baud_rate;
23     signal sending_done : std_logic := '0';
24     signal button_detected : std_logic := '0';
25
26 begin
27     process(clk) is
28         variable cnt_period : integer := 0;
29         variable start_bit_done : std_logic := '0';
30         variable char_bits_done : std_logic := '0';
31     begin
32         if rising_edge(clk) then
33
34             if rst = '1' then
35                 state <= IDLE;
36                 out_bit <= '1';
37             else
38
39                 if button_detected = '1' then
40                     state <= SENDING;
41                     button_detected <= '0';
42                 elsif sending_done = '1' then
43                     state <= IDLE;
44                     sending_done <= '0';
45                 end if;
46
47                 case state is
48                     when IDLE =>
49                         if we = '1' then
50                             button_detected <= '1';

```



```

51         else
52             out_bit <= '1';
53         end if;
54         when SENDING =>
55             if start_bit_done = '0' then
56                 out_bit <= '0';
57                 if cnt_period < bit_period then
58                     cnt_period := cnt_period + 1;
59                 else
60                     cnt_period := 0;
61                     start_bit_done := '1';
62                 end if;
63             elsif char_bits_done = '1' then
64                 out_bit <= '1';
65                 if cnt_period < bit_period then
66                     cnt_period := cnt_period + 1;
67                 else
68                     cnt_period := 0;
69                     char_bits_done := '0';
70                     start_bit_done := '0';
71                     sending_done <= '1';
72                 end if;
73             else
74                 if symbol_cnt < 8 then
75
76                     out_bit <= byte_in(symbol_cnt);
77                     cnt_period := cnt_period + 1;
78
79                     if cnt_period > (bit_period - 1 ) then
80                         symbol_cnt <= symbol_cnt + 1;
81                         cnt_period := 0;
82                     end if;
83                 else
84                     symbol_cnt <= 0;
85                     char_bits_done := '1';
86                 end if;
87             end if;
88
89         end case;
90
91     end if;
92
93     end if;
94 end process;
95 end architecture uart_tx_a;

```

Vor dem Case-Statement wird geprüft ob eine der Zustandsübergangskriterien erfüllt ist. Wenn der we-Eingang aktiv ist, wird das Zustandsübergangssignal button_detected auf '1' gesetzt, wodurch der Übergang von IDLE zu SENDING ausgelöst wird. Sobald

das Senden abgeschlossen ist, wird das Zustandsübergangssignal `sending_done` auf '1' gesetzt, wodurch der Übergang von SENDING zu IDLE ausgelöst wird.

```

1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity uart_tx_tb is
6  end entity;
7
8
9  architecture uart_tx_tb_a of uart_tx_tb is
10     signal clk : std_logic := '0';
11     signal byte_in : std_logic_vector(7 downto 0) := (others => '0');
12     signal we : std_logic := '0';
13     signal rst : std_logic := '0';
14     signal out_bit : std_logic := '0';
15
16 begin
17     uart_tx_inst : entity work.uart_tx
18     port map(
19         clk=> clk,
20         byte_in => byte_in,
21         we => we,
22         rst => rst,
23         out_bit => out_bit
24     );
25
26     clk <= not clk after 83.33 ns / 2;
27
28     process
29     begin
30         -- Reset
31         rst <= '1';
32         wait for 100 ns;
33         rst <= '0';
34
35         -- Check IDLE
36         wait for 50 ns;
37         assert out_bit = '1' report "ERROR: out_bit not idle high after reset"
38             ↪ severity error;
39
40         -- 1
41         byte_in <= "11010001";
42         wait for 100 ns;
43         we <= '1';
44         wait for 100 ns;
45         we <= '0';
46
47         wait for 50 us;
48         assert out_bit = '0'
49             report "ERROR: Startbit not detected (Test 1)"
50                 severity warning;

```

```

50
51     wait for 2 ms;
52     assert out_bit = '1'
53         report "ERROR: Line not back to idle (Test 1)"
54             severity error;
55
56     -- 2
57     byte_in <= "10101010";
58     wait for 100 ns;
59     we <= '1';
60     wait for 300 ns;
61     we <= '0';
62
63     wait for 50 us;
64     assert out_bit = '0'
65         report "ERROR: Startbit not detected (Test 2)"
66             severity warning;
67
68     wait for 2 ms;
69
70     -- 3
71     byte_in <= "11110000";
72     wait for 100 ns;
73     we <= '1';
74     wait for 300 ns;
75     we <= '0';
76
77     wait for 50 us;
78     assert out_bit = '0'
79         report "ERROR: Startbit not detected (Test 3)"
80             severity warning;
81
82     wait for 2 ms;
83
84     -- 4
85     byte_in <= "00001111";
86     wait for 100 ns;
87     we <= '1';
88     wait for 300 ns;
89     we <= '0';
90
91     wait for 50 us;
92     assert out_bit = '0'
93         report "ERROR: Startbit not detected (Test 4)"
94             severity warning;
95
96     wait for 2 ms;
97
98     -- 5
99     byte_in <= "00110011";

```

```

100     wait for 100 ns;
101     we <= '1';
102     wait for 300 ns;
103     we <= '0';
104
105     wait for 50 us;
106     assert out_bit = '0'
107         report "ERROR: Startbit not detected (Test 5)"
108             severity warning;
109
110     wait for 2 ms;
111
112     -- Ende
113     assert false report "Simulation finished successfully" severity note;
114     wait;
115
116
117     end process;
118 end architecture uart_tx_tb_a;

```

Die Testbench testet fünf mal die Funktionsalität des TX Moduls. Es werden jeweils verschiedene Byte-Werte gesendet. Nach jedem Senden wird überprüft, ob das Startbit korrekt erkannt wurde und ob die Leitung nach dem Senden wieder auf den Idle-Zustand zurückkehrt. Die Asserts dienen zur Fehlererkennung. Schlägt keines der Asserts fehl, so wird am Ende der Simulation eine Erfolgsmeldung ausgegeben. Die folgenden drei Bilder zeigen drei der Tests aus der Simulation. Zu sehen ist, dass die Variable `symbol_cnt` von 0 bis 7 hochzählt und, dabei die entsprechenden Bits des `byte_in`-Signals auf das `out_bit`-Signal überträgt. Auch die Funktionalität der Start- und Stoppbits ist gut zu erkennen.

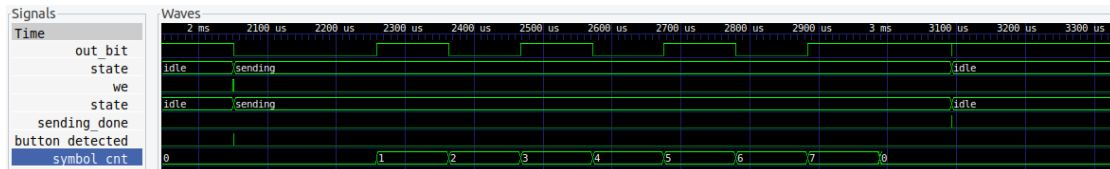


Abbildung 7: UART TX Simulation 1

Testcase 2 zeigt die Übertragung des Byte-Werts "10101010".

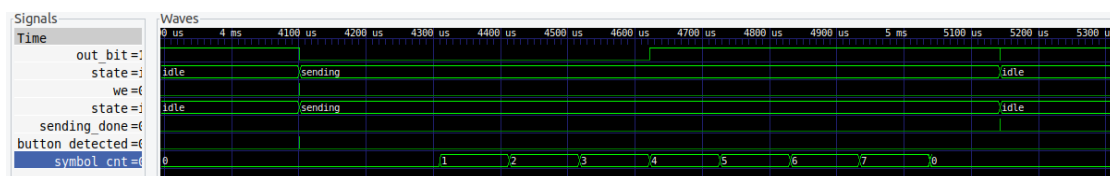


Abbildung 8: UART TX Simulation 2

Testcase 3 zeigt die Übertragung des Byte-Werts "11110000".

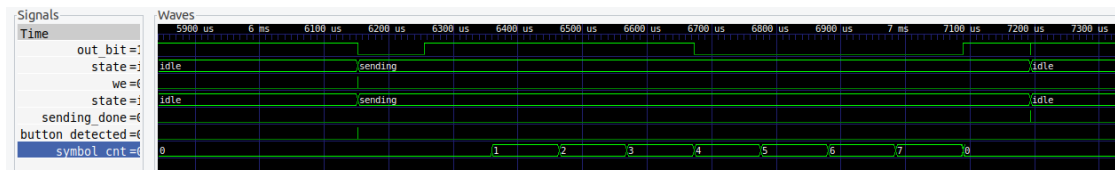


Abbildung 9: UART TX Simulation 3

Testcase 4 zeigt die Übertragung des Byte-Werts "00001111".

3 Aufgabe 4: UART RX

Das folgende Blockdiagramm zeigt die Struktur des UART RX Moduls. Es besteht aus einem Hauptmodul `uart_rx` und einem Testbench-Modul `uart_rx_tb`. Das Hauptmodul implementiert die UART RX Funktionalität: Es empfängt serielle Daten über den `uart_in`-Eingang, synchronisiert das eingehende Signal, sampled die Datenbits in der Mitte des Bitfensters und schaltet die LED je nach empfangenem Zeichen ein oder aus. Das Testbench-Modul dient zur Simulation und Verifikation der Funktionalität des Hauptmoduls.

Das Modul arbeitet mit einer Baudrate von 9600 bei einem 12 MHz Systemtakt (8N1-Konfiguration: 8 Datenbits, kein Paritätsbit, 1 Stoppbit). Empfängt das Modul das Zeichen 'j' (ASCII 106, 0x6A), wird die LED eingeschaltet. Empfängt es das Zeichen 'k' (ASCII 107, 0x6B), wird die LED ausgeschaltet. Alle anderen empfangenen Zeichen werden ignoriert und der LED-Zustand bleibt unverändert.

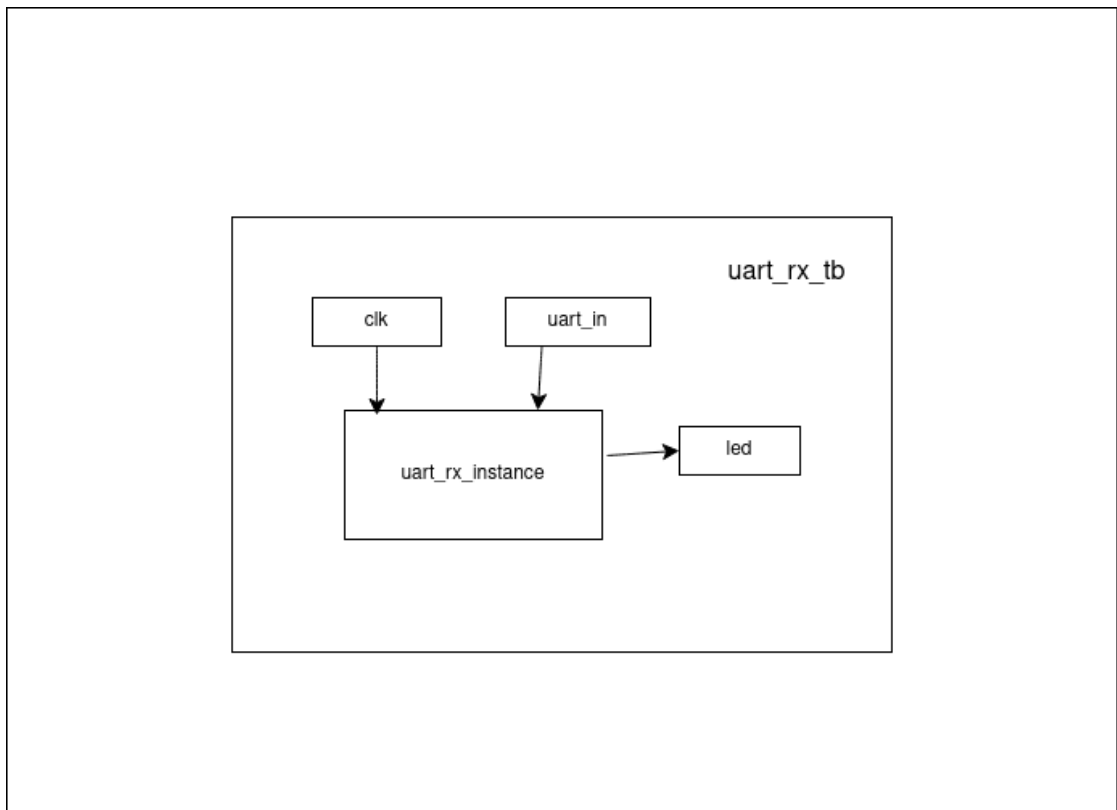


Abbildung 10: UART RX Blockdiagramm

Die Statesmaschine des UART RX Moduls besteht aus zwei Hauptzuständen: **IDLE** und **READING**. Der Automat ist deterministisch.

Im Zustand **IDLE** wartet das Modul auf eine fallende Flanke am **uart_in**-Signal, die das Startbit signalisiert. Durch die Zweiflanken-Synchronisation (**uart_sync**) werden metastabile Zustände vermieden. Bei erkannter fallender Flanke ("10") wechselt der Automat in den Zustand **READING** und initialisiert alle Zähler.

Im Zustand **READING** wartet das Modul zunächst eine halbe Bitperiode (**pre_cnt**), um das Sampling in die Mitte des ersten Datenbits zu verschieben. Anschließend werden 8 Datenbits eingelesen, wobei jeweils nach einer vollen Bitperiode (**clk_cnt**) gesampelt wird. Nach dem letzten Datenbit wird noch eine halbe Bitperiode gewartet (**post_cnt**), um das Stoppbit zu überspringen. Danach wird der empfangene Wert im **cache** mit den ASCII-Codes für 'j' und 'k' verglichen und die LED entsprechend geschaltet.


```

1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity uart_rx is
6      port(
7          clk : in std_logic;
8          uart_in : in std_logic;
9          led : out std_logic
10     );
11     end entity;
12
13 architecture uart_rx_arch of uart_rx is
14
15     signal cache : std_logic_vector(7 downto 0) := (others => '0');
16     signal led_signal : std_logic := '0';
17     signal uart_sync : std_logic_vector(1 downto 0) := (others => '1');
18     constant baud_rate : integer := 9600;
19     constant clk_freq : integer := 12000000;
20     constant bit_period : integer := clk_freq / baud_rate - 1;
21     constant post_cnt_max : integer := (3 * bit_period) / 2;
22     type state_type is (IDLE, READING);
23     signal state : state_type := IDLE;
24
25 begin
26
27     process(clk)
28         variable pre_cnt : integer range 0 to bit_period / 2 := 0;
29         variable post_cnt : integer range 0 to (3 * bit_period) / 2 := 0;
30         variable bit_cnt : integer range 0 to 8 := 0;
31         variable clk_cnt : integer range 0 to bit_period := 0;
32
33     begin
34         if rising_edge(clk) then
35
36             uart_sync <= uart_sync(0) & uart_in; -- Shift in the new bit for
37             ↳ synchronization
38
39             if state = IDLE then
40
41                 if uart_sync(1 downto 0) = "10" then -- Detect the falling edge
42                     ↳ for start bit
43                     state <= READING;
44                     pre_cnt := 0;
45                     post_cnt := 0;
46                     bit_cnt := 0;
47                     clk_cnt := 0;
48                     cache <= (others => '0');
49                 end if;

```

```

49         elsif state = READING then
50
51             if pre_cnt < (bit_period / 2) then
52
53                 pre_cnt := pre_cnt + 1;
54
55             elsif bit_cnt < 8 then
56
57                 if clk_cnt < bit_period then
58
59                     clk_cnt := clk_cnt + 1;
60
61                 else
62
63                     cache(bit_cnt) <= uart_sync(0); -- Sample the bit
64                     bit_cnt := bit_cnt + 1;
65                     clk_cnt := 0;
66
67                 end if;
68
69             elsif post_cnt < (post_cnt_max) then
70
71                 post_cnt := post_cnt + 1;
72
73             else
74
75                 if unsigned(cache) = 106 then --checks if the received
76                     ⇨ byte is 106 (ASCII for 'j')
77                     led_signal <= '1';
78                 elsif unsigned(cache) = 107 then --checks if the received
79                     ⇨ byte is 107 (ASCII for 'k')
80                     led_signal <= '0';
81                 end if;
82                 state <= IDLE;
83
84             end if;
85         end if;
86     end if;
87 end process;
88
89 led <= led_signal;
90 end architecture uart_rx_arch;

```

Der VHDL-Code implementiert den beschriebenen Empfänger. Besonders wichtig sind:

- **Zweiflanken-Synchronisation** (Zeile 30): `uart_sync <= uart_sync(0) & uart_in` verhindert Metastabilität durch asynchrone Eingangssignale.
- **Startbiterkennung** (Zeile 34): Die fallende Flanke "10" am synchronisierten Si-

gnal löst den Empfang aus.

- **Bit-Sampling in der Mitte des Fensters:** `pre_cnt` verschiebt das erste Sampling um eine halbe Bitperiode, `clk_cnt` sorgt für die korrekte Timing der folgenden Bits.
- **Zeichenerkennung** (Zeile 56–59): Nach vollständigem Empfang wird der `cache` mit 106 (`'j'`) und 107 (`'k'`) verglichen.

```

1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4  use std.env.all;
5
6  entity uart_rx_tb is
7  end entity;
8
9  architecture uart_rx_tb_arch of uart_rx_tb is
10
11     signal clk : std_logic := '0';
12     signal uart_in : std_logic := '1';
13     signal led : std_logic;
14
15     constant baud_rate : integer := 9600;
16     constant clk_freq : integer := 12000000;
17     constant clk_periode : time := 83.33 ns;
18     constant baud_period : time := 1 sec / baud_rate;
19
20 begin
21
22     uart_rx_inst : entity work.uart_rx
23     port map(
24         clk => clk,
25         uart_in => uart_in,
26         led => led
27     );
28
29
30     clk <= not clk after clk_periode/2; -- 12 MHz clock
31
32     process
33         variable byte_jump : std_logic_vector(7 downto 0) := "01101010"; --
34         ↪ ASCII for 'j'
35         variable byte_kill : std_logic_vector(7 downto 0) := "01101011"; --
36         ↪ ASCII for 'k'
37         variable byte_other : std_logic_vector(7 downto 0) := "11111111"; -- A
38         ↪ byte that is not 'j' or 'k'
39     begin
40         wait for 5000 ns; -- Wait for the system to stabilize
41
42         -- Send start bit
43         uart_in <= '0';
44         wait for baud_period; -- Wait for one bit period (1/baud_rate)
45
46         -- Send data bits (LSB first)
47         for i in 0 to 7 loop
48             uart_in <= byte_jump(i);
49             wait for baud_period; -- Wait for one bit period
50         end loop;

```

```

48
49      -- Send stop bit
50      uart_in <= '1';
51      wait for baud_period; -- Wait for one bit period
52
53      wait for 5000 ns; -- Wait before ending the simulation
54
55      assert led = '1' report "LED should be ON after receiving 'j'" severity
56      ↪ error;
57
58      wait for 500000 ns; -- Wait before sending the next byte
59
60      -- Send byte for 'k'
61      uart_in <= '0';
62      wait for baud_period; -- Wait for one bit period (1/baud_rate)
63
64      -- Send data bits (LSB first)
65      for i in 0 to 7 loop
66          uart_in <= byte_kill(i);
67          wait for baud_period; -- Wait for one bit period
68      end loop;
69
70      -- Send stop bit
71      uart_in <= '1';
72
73      wait for baud_period; -- Wait for one bit period
74
75      wait for 5000 ns; -- Wait before ending the simulation
76
77      assert led = '0' report "LED should be OFF after receiving 'k'"
78      ↪ severity error;
79
80      wait for 500000 ns; -- Wait before sending the next byte
81
82      -- test with a different byte (not 'j' or 'k')
83      uart_in <= '0';
84
85      wait for baud_period; -- Wait for one bit period (1/baud_rate)
86
87      -- Send data bits (LSB first)
88      for i in 0 to 7 loop
89          uart_in <= byte_other(i); -- Send a byte that is not 'j' or 'k'
90          wait for baud_period; -- Wait for one bit period
91      end loop;
92
93      -- Send stop bit
94      uart_in <= '1';
95
96      wait for baud_period; -- Wait for one bit period

```

```

96     wait for 50000 ns; --
97
98     assert led = '0' report "LED should remain OFF after receiving a byte
    ↪   other than 'j' or 'k'" severity error;
99
100    report "Simulation finished successfully";
101    stop(0); -- Clean exit, no error code
102    end process;
103
104 end architecture;

```

Die Testbench verifiziert die korrekte Funktionalität des UART RX Moduls mit drei Testfällen:

1. **Testfall 1 – LED einschalten:** Es wird das Zeichen 'j' (ASCII 106, 01101010) gesendet. Das Startbit (0), gefolgt von den 8 Datenbits (LSB first) und dem Stoppbit (1) werden seriell über `uart_in` übertragen. Nach Abschluss des Empfangs wird mit einem Assert geprüft, ob `led = '1'` ist.
2. **Testfall 2 – LED ausschalten:** Es wird das Zeichen 'k' (ASCII 107, 01101011) gesendet. Der Ablauf ist identisch zu Testfall 1. Das Assert prüft, ob `led = '0'` ist.
3. **Testfall 3 – Ungültiges Zeichen:** Es wird das Zeichen 0xFF (11111111) gesendet, das weder 'j' noch 'k' entspricht. Das Assert prüft, dass die LED ihren vorherigen Zustand ('0') beibehält.

Alle Testfälle verwenden die korrekte Baudrate von 9600 Bd (Bitperiode $\sim 104,17 \mu\text{s}$) und den 12 MHz Systemtakt. Die Asserts dienen zur automatisierten Fehlererkennung: Schlägt keines fehl, so ist die Grundfunktionalität des Empfängers verifiziert.



Abbildung 11: GTKWave-Simulation des UART RX Moduls: Verlauf der Signale `state`, `uart_in` und `led`

Die Simulation zeigt den vollständigen Ablauf der drei Testfälle. Von oben nach unten sind die Signale `state` (Zustandsautomat), `uart_in` (serieller Dateneingang) und `led` (Ausgang) dargestellt.

Im Verlauf von `uart_in` sind die drei Übertragungen erkennbar: Zunächst das Zeichen 'j' (Startbit, gefolgt von 01101010, Stoppbit), anschließend 'k' (01101011) und zuletzt 0xFF (11111111). Das `state`-Signal wechselt jeweils von IDLE zu READING während des Empfangs und zurück zu IDLE nach Abschluss. Das `led`-Signal zeigt den erwarteten Verlauf: Es geht nach Empfang von 'j' auf '1', fällt nach Empfang von 'k' auf '0' und bleibt bei ungültigem Zeichen unverändert.